

Research Paper Section: Framework, Model, Task Automation, Architecture & Features

This document is a drafting artifact for the Phase 1 research paper. It combines the agentic-framework explanation and selection, the AI model selection, the accountant/CA task → AI automation mapping, the system architecture, and the feature list. Written to be copy-pasted into the final research paper PDF with minimal edits.

1. How AI Agents / Agentic Frameworks Work

An AI agent is a system built around a large language model (LLM) that doesn't just respond to a single prompt, but can reason, plan, call external tools, retain context across steps, and take multi-step actions to complete a task autonomously. Unlike a simple chatbot, an agent typically follows a loop: it receives a goal, decides what action or tool to use, executes that action (e.g., querying a database, calling an API, running a calculation), observes the result, and repeats until the goal is achieved. Agentic frameworks provide the scaffolding for this — handling tool-calling, memory/state management, multi-agent coordination (handoffs between specialized agents), and guardrails to keep the agent's behavior safe and predictable.

For an accounting assistant, this means an agent can interpret a request like "generate the P&L for March," decide it needs to query the ledger database, fetch the relevant entries, run calculations, and return a structured report — all without the developer hardcoding every possible query pattern.

2. Agentic Framework Selection

Framework selected: OpenAI Agents SDK. As of 2026, it has matured from OpenAI's earlier experimental "Swarm" project into a production-grade SDK with sandbox execution and a built-in harness system, making it a current, well-supported industry standard rather than a research prototype. It is widely recognized as the fastest path from zero to a working agent when a team is committed to OpenAI's models, offering handoff patterns and guardrails that let developers build reliable multi-step workflows in relatively little code. Since this project's AI layer is built entirely on OpenAI's models, the SDK's tight native integration removes the need for extra abstraction layers, resulting in faster development and simpler debugging — both valuable given the project timeline.

Comparison with alternatives

Framework	Strengths	Trade-offs
OpenAI Agents SDK	Fastest to build with, native tool-calling/guardrails, tightly integrated with OpenAI models	Vendor-locked to OpenAI
LangChain / LangGraph	Most flexible, model-agnostic, largest ecosystem (110K+ GitHub stars), best for complex stateful workflows	Steeper learning curve, more abstraction/boilerplate
CrewAI	Best for role-based multi-agent collaboration, most approachable for beginners, strong MCP tool support	Less suited when the task doesn't naturally split into multiple agent "roles"

LangChain (and its stateful counterpart, LangGraph) is model-agnostic and favored for complex, stateful workflows requiring fine-grained control over conditional logic, error recovery, and human-in-the-loop checkpoints — but this power comes at the cost of a steeper learning curve and more boilerplate, historically criticized for having too many abstraction layers. CrewAI organizes agents into "crews" with defined roles (e.g., a bookkeeping agent, an audit agent, a reporting agent) that collaborate on a shared task, making it compelling for projects that naturally split into specialized "accountant" sub-agents.

Why OpenAI Agents SDK was ultimately chosen: Given the project's single-model commitment (OpenAI), limited timeline, and the need to move quickly from prototype to a working demo, the OpenAI Agents SDK offered the best trade-off between simplicity and capability. It avoids the abstraction overhead of LangChain/LangGraph and doesn't require designing a full multi-agent "crew" structure the way CrewAI does — while still supporting tool-calling, guardrails, and handoffs, which are sufficient for the accounting automation tasks required in this assignment (natural-language entry creation, report generation, anomaly detection, and Q&A). LangChain/LangGraph would be the stronger choice for a larger, model-agnostic enterprise system, and CrewAI would suit a project explicitly built around multiple specialized agents — but for this assignment's scope, the OpenAI Agents SDK provides the fastest, most maintainable path to a working, reliable product.

Practical implication carried into the design: the system is built as **one agent with a scoped toolset** (query ledger, run aggregation, write journal entry, flag anomaly, run reconciliation match) rather than a crew of specialized agents — enabled directly by the OpenAI Agents SDK's handoff/guardrail model, and reflected throughout the automation mapping in Section 4.

3. AI Model Selection

For the AI layer of this application, **GPT-4o mini** was selected as the underlying language model powering the agent, for the following reasons:

- **Cost-efficiency:** GPT-4o mini is priced at approximately \$0.15 per million input tokens and \$0.60 per million output tokens (with cached input as low as \$0.075/million) — a fraction of the cost of larger models like GPT-4o (\$2.50/\$10.00 per million tokens). For a project of this scale, this keeps running costs negligible, often just a few dollars a month even under regular usage.
- **Sufficient capability for the task:** Accounting automation tasks in this project — entry classification, natural-language querying, report generation, summarization, basic anomaly flagging — are structured and don't require frontier-level reasoning. GPT-4o mini scores well on general benchmarks (82% MMLU) and even outperforms the original GPT-4 on chat-preference leaderboards, making it more than capable here while staying lightweight and fast.
- **Feature support:** It supports a 128K token context window, up to 16K tokens of output, and native function-calling — essential since the AI agent must call backend tools (e.g., "fetch March expenses," "generate P&L"), which is a core requirement of the OpenAI Agents SDK architecture.
- **Ecosystem fit:** Since the project uses the OpenAI Agents SDK, an OpenAI-native model avoids cross-provider integration complexity and keeps the entire AI layer within one ecosystem.
- **Free-tier friendliness:** OpenAI's low per-token cost and playground/testing credits make GPT-4o mini practical to prototype and test extensively without budget concerns, satisfying the assignment's "free-tier friendly" preference.

4. Core Accountant / CA Responsibilities (by frequency)

Before mapping automation methods, it's worth grounding the task list in what accountants and CAs actually do. Their core functions span five areas — bookkeeping, financial reporting, tax preparation, auditing, and financial analysis/advisory — carried out across daily, weekly, monthly, and annual cycles:

- **Daily:** recording transactions, coding invoices/receipts, checking cash position, journal entries, bank feed review.
- **Weekly:** documenting receipts, organizing supporting files, reviewing unpaid bills/receivables.
- **Monthly:** bank/vendor reconciliation, payroll processing, month-end close (adjusting entries, verifying balances), trial balance, management reports.
- **Quarterly/Annual:** P&L statement, balance sheet, cash flow statement, statutory/tax filings, financial audits, budgeting, financial ratio analysis and advisory.

5. Task → AI Automation Mapping

Legend for "AI-Automatable?":  Fully automatable today ·  Automatable with mandatory human sign-off (regulatory/financial risk) ·  Automatable as deterministic

computation, with the LLM only orchestrating/narrating (not generating numbers itself).

5.1 Data Capture & Recording

Task	Frequency	AI-Automatable?	Concrete AI Automation Method	Human-in-the-loop notes
Expense/receipt logging	Daily	✓	Hybrid OCR + LLM extraction: OCR pulls raw text off the receipt/invoice image, then the LLM structures it into fields (vendor, amount, date, category) via function-calling. This hybrid approach handles layout variability that pure template-based OCR fails on.	Low-confidence extractions routed to a review queue.
Natural-language entry creation	Daily	✓	User says "log \$40 for office supplies yesterday" → LLM parses intent and calls a <code>create_transaction</code> tool with structured arguments (amount, date, category), rather than generating the ledger row as free text.	Agent confirms parsed fields back to the user before committing.
Invoice data entry (AP)	Daily/Weekly	✓	Same OCR+LLM hybrid extraction pipeline as receipts, matched against vendor master data via fuzzy/embedding lookup.	Exceptions (new vendor, unusual amount) flagged for approval.

5.2 Ledgers & Journal Entries

Task	Frequency	AI-Automatable?	Concrete AI Automation Method	Human-in-the-loop notes
Chart-of-accounts coding/categorization	Daily	✓	LLM classification with a confidence score, using historical categorization as few-shot context.	Below-threshold confidence → human review; corrections feed back as future few-shot examples.
Double-entry journal posting	Daily	✓ (tool-mediated)	Agent calls a scoped <code>post_journal_entry</code> tool against the ledger API — the LLM decides <i>what</i> entry to post from natural language, but the debit/credit posting itself is deterministic application logic, not LLM-generated text.	Guardrails restrict which accounts/amount ranges an agent can post without explicit confirmation.

5.3 Reconciliation

Task	Frequency	AI-Automatable?	Concrete AI Automation Method	Human-in-the-loop notes
Bank/vendor reconciliation	Monthly (or continuous)	✓	Embedding/fuzzy-match on amount+date+description to auto-match transactions to bank feed lines; LLM adjudicates ambiguous near-matches and explains its reasoning.	Unmatched items surfaced in a review queue rather than silently resolved.

5.4 Reporting (Trial Balance, P&L, Balance Sheet, Cash Flow)

Task	Frequency	AI-Automatable?	Concrete AI Automation Method	Human-in-the-loop notes
Trial balance	Monthly	⚙️	Deterministic SQL/pandas aggregation over the ledger, triggered by the agent as a tool call; LLM narrates any imbalance found.	—
Profit & Loss statement	Monthly/Quarterly	⚙️	Agent resolves "generate the P&L for March" into a scoped date-range aggregation query (revenue – expenses by category), executed as deterministic code; LLM formats and narrates the result (e.g., notable variances vs. prior month).	Numbers always come from the computation layer, never generated directly by the LLM — this distinction is a core design principle for reliability.
Balance sheet	Monthly/Quarterly	⚙️	Same orchestration pattern: agent calls an <code>assets/liabilities/equity</code> aggregation tool, LLM narrates.	—
Cash flow statement	Monthly/Quarterly	⚙️	Derived from transaction-level cash movements via deterministic categorization (operating/investing/financing) rather than LLM inference.	—

5.5 Audit & Anomaly Detection

Task	Frequency	AI-Automatable?	Concrete AI Automation Method	Human-in-the-loop notes
Monthly/ad-hoc audit of entries	Monthly/On-demand	●	Unsupervised ML anomaly detection (e.g., Isolation Forest, clustering) run across all journal entries for the period — not sampling-based like manual audits, since ML can evaluate the entire dataset. Real-world precedent: EY's Helix GL Anomaly Detector and MindBridge's AI-powered anomaly detection both apply this pattern in production audit tooling. The LLM layer then explains <i>why</i> a flagged entry looks anomalous in plain language and drafts a suggested corrective note.	Flags are surfaced for human review, not auto-corrected — audits carry regulatory/reputational risk.
Fraud pattern detection	Continuous	●	Supervised/unsupervised models trained on transaction patterns (vendor behavior, timing, amount clustering) flag deviations from normal financial behavior.	Same as above — investigation and final judgment stays human.

5.6 Tax & Compliance Summaries

Task	Frequency	AI-Automatable?	Concrete AI Automation Method	Human-in-the-loop notes
Tax summary preparation	Quarterly/Annual	●	Retrieval-Augmented Generation (RAG) over the ledger data plus relevant tax-rule reference documents to draft a summary of taxable income/deductions.	Mandatory human/CA sign-off before filing — regulatory risk is too high for unsupervised automation.

5.7 Analysis & Advisory / Natural-Language Q&A

Task	Frequency	AI-Automatable?	Concrete AI Automation Method	Human-in-the-loop notes
"How much did we spend on utilities in March?"	On-demand	✓	Text-to-SQL / RAG over the transaction database: the agent translates the NL question into a scoped, read-only aggregation query, executes it, and narrates the number.	Read-only tool scope — no write access needed for Q&A.
Spending pattern summaries	Monthly	✓	Agent runs aggregation/grouping queries (by category, vendor, time) and the LLM narrates trends ("marketing spend rose 18% vs. last month").	—
Budget forecasting	Quarterly/Annual	✓	Time-series/statistical forecasting model (e.g., simple trend/seasonal decomposition) over historical transaction data, with the LLM narrating the projection and its assumptions.	Forecasts labeled as estimates, not guarantees.

5.8 Design Principle Carried Into the Architecture

Across every row above, the same pattern recurs and should be stated explicitly in the paper's architecture section: **the LLM is the orchestrator and narrator, not the calculator**. All numeric outputs (balances, totals, matches) come from deterministic tool calls (SQL/pandas/ledger API) invoked by the agent; the LLM's job is to understand intent, choose the right tool/query, and explain the result in natural language. This keeps the system auditable and avoids the core risk of LLM-generated financial figures (hallucinated numbers). This is a direct consequence of the OpenAI Agents SDK's tool-calling/guardrail model chosen in Section 2.

6. System Architecture

The system follows a modular, containerized architecture with four layers: **Frontend, Backend/API, Database, and AI Layer**.

1. Frontend — Next.js (TypeScript) Provides the web UI for managing financial records and interacting with the AI assistant via a chat interface. Communicates with the backend via REST API calls (JSON over HTTPS). TypeScript keeps request/response types in sync with backend Pydantic schemas.

2. Backend & API Layer — Python + FastAPI (managed with uv) FastAPI handles routing and async performance, with **uv** managing dependencies/environments for fast, reproducible builds. All request/response payloads are validated via **Pydantic models**, ensuring strict validation of financial data (amounts, dates, categories). Exposes REST endpoints for: - CRUD on financial entries (expenses, income, office costs) - Report generation (P&L, balance sheet, trial

balance, audit summary) - A chat/agent endpoint forwarding natural-language requests to the AI layer

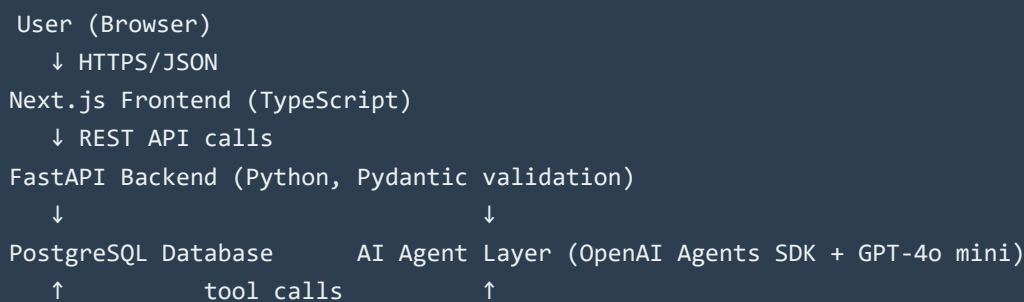
3. Database — PostgreSQL Stores ledger entries, categories, monthly/yearly records, and audit logs. Connected via an ORM (e.g., SQLAlchemy), with Pydantic models mirroring the schema for consistent validation. The relational structure fits accounting data well (entries, categories, reports naturally relate).

4. AI Layer — OpenAI Agents SDK + GPT-4o mini Sits behind FastAPI as a callable service. When a user sends a natural-language request (e.g., *"How much did we spend on utilities in March?"*), FastAPI routes it to the agent, which: - Interprets intent (query, report generation, entry creation, audit, etc.) - Calls backend "tool" functions that query/write to PostgreSQL - Returns a structured response back through FastAPI to the Next.js frontend

The agent never touches the database directly — it only goes through backend-exposed tools, keeping validation and business logic centralized in FastAPI.

5. Containerization — Docker Separate containers for Next.js frontend, FastAPI backend, and PostgreSQL, orchestrated via `docker-compose`, ensuring consistent environments across development and deployment.

Communication Flow:



7. Features to Implement

1. Core Bookkeeping (CRUD) - Add/edit/delete daily expense entries - Add/edit/delete income entries - Add/edit/delete monthly office expenses - Categorize entries (utilities, rent, salaries, supplies, etc.)

2. AI-Powered Natural Language Entry - Add entries via natural language (e.g., *"Add Rs5,000 electricity bill for July"*) - AI auto-categorizes uncategorized entries based on description

3. Financial Reports (AI-generated) - Profit & Loss (P&L) statement for any given month/period - Balance Sheet generation - Trial Balance generation - Cash Flow summary

4. Query & Insights (Natural Language Q&A) - Answer questions like *"How much did we spend on utilities in March?"* - Spending pattern summaries (top expense categories per quarter) -

Month-over-month comparison insights

5. Audit & Anomaly Detection - Run an automated audit for any given month - Flag anomalies (duplicate entries, unusually large transactions, missing categories) - Reconciliation checks (entries vs. expected totals)

6. Tax/Compliance Summary (basic) - Generate a simplified tax-relevant summary (taxable income/expenses) for a period

7. Dashboard & History - Visual dashboard (charts) of income vs. expense trends - Searchable transaction history with filters (date, category, amount range)

8. References

Big Four / Industry AI-in-Accounting Reports - [EY — AI in Finance: The Hidden Advantage for Tech Companies](#) - [EY — Tax Accounting Transformation with AI and Automation](#) - [EY — How an AI application can help auditors detect fraud](#) - [Accounting Today — EY Assurance Professionals Get Access to AI Agents](#) - [Accounting Today — AI Thought Leaders Survey 2026: Process Predictions](#)

Anomaly Detection & Audit Automation - [MindBridge — Anomaly Detection Techniques](#) - [MindBridge — AI-Powered Anomaly Detection: Going Beyond the Balance Sheet](#)

Academic Research - [arXiv — AuditCopilot: Leveraging LLMs for Fraud Detection in Double-Entry Bookkeeping \(2512.02726\)](#) - [arXiv — Continual Learning for Unsupervised Anomaly Detection in Continuous Auditing of Financial Accounting Data \(2112.13215\)](#)

Industry / Implementation Blogs - [Intuz — AI in Accounting: 7 Use Cases, Examples and Tools \(2026\)](#) - [Intuz — AI Workflow Automation: Use Cases & Benefits](#) - [Satva Solutions — Top 10 AI Accounting Use Cases \(2026\)](#) - [Satva Solutions — How I Automated Accounting for SyncTools Using an AI Agent](#) - [Satva Solutions — Ethical and Regulatory Challenges of Using AI in Accounting](#) - [Dualentry — AI in Accounting: The Complete 2026 Guide](#)

OCR vs. LLM Document Extraction - [Vellum — Document Data Extraction in 2026: LLMs vs OCRs](#) - [Mindee — LLMs vs OCR APIs for Document Processing](#) - [Klipa — LLMs vs OCR Data Extraction](#) - [SparkReceipt — OCR Accounting: Say Goodbye to Manual Data Entry](#) - [Raftlabs — OCR vs LLM: How We Built Automated Invoice Scanning](#) - [Tvarana — LLM-Based AI OCR for Vendor Invoices](#)

Accountant / CA Duties - [ACCA — What Do Accountants Do?](#) - [ProfitBooks — The 5 Essential Duties of an Accountant \(2026 Guide\)](#) - [CourseCareers — Daily Tasks of Entry-Level Accountants](#)

Agentic Framework Comparison - [Atlan — OpenAI Agents SDK vs LangChain vs CrewAI: 2026 Guide](#) - [LangChain — The Best AI Agent Frameworks in 2026](#) - [Valueaddvc — AI Agent Frameworks 2026: LangChain, CrewAI, AutoGen, and OpenAI Agents Compared](#) - [Uvik — Agentic](#)

[AI Frameworks 2026: Production Comparison - PE Collective — AI Agent Frameworks Compared: LangGraph vs CrewAI vs AutoGen \(2026\) - Cordum — Best AI Agent Frameworks 2026: LangChain, CrewAI, AutoGen - Medium \(ATNO for GenAI & Agentic AI\) — 10 AI Agent Frameworks You Should Know in 2026](#)

AI Model Pricing - [OpenRouter — GPT-4o mini: API Pricing & Benchmarks](#)